

GROUP 9
BREAKING TASK
ISOLATION
(PROJECT #2)

BRIEF INTRODUCTION

TASK ISOLATION IN RTOSes



WHAT IS IT?



WHY IS IT IMPORTANT?



WHAT RISKS ARE THERE?



WHAT SOLUTIONS CAN
WE APPLY TO
INCREASE SECURITY?

INITIAL ASSUMPTIONS

- INSIDE ATTACKER THREAT MODEL
- FLAT MEMORY MODEL
- HOMOGENEOUS PRIVILEGE
- GLOBAL DATA VISIBILITY
- STACK INTROSPECTION



SUCH ASSUMPTIONS
WILL FALL AFTER A
SPECIFIC PROTECTED
IMPLEMENTATION

GENERAL OVERVIEW OF THE REPOSITORY

FOLDER #1: DEMO

/MAIN.C (VULNERABLE TASKS)

/FREERTOSCONFIG.H

/STARTUP.C

/UART.C

/MPS2_M3.LD

/MAKEFILE

FOLDER #2: DEMO_MPU

/MAIN.C (SECURE TASKS)

/FREERTOSCONFIG.H (WITH MPU CONFIGURATION)

/STARTUP.C (WITH MPU EXCEPTION HANDLER)

/UART.C

/MPS2_M3.LD (WITH ADDED DATA SECTION)

/MAKEFILE (WITH MPU SUPPORT)

DOCUMENTATION FILES:

→ /DOCUMENTATION.PDF
→ /README.MD



THE OVERALL PROCESS FLOW

PRIORITYAND
STACKATTACKER



CHANGES ITS OWN
PRIORITY FROM 1 TO 7,
FINDS THE LONGEST
PRINTABLE STRING IN
TARGET'S STACK AND
RANDOMLY MODIFIES IT

TARGET



HOLDS THE KEY
"PCSECRET[]" AND CHECKS
FOR ANY POSSIBLE
MODIFICATION, SAVING ITS
COUNTER GLOBALLY

GLOBALVAR
ATTACKER



ATTACKS THE GLOBAL
VARIABLE USED BY
TARGET, CHANGING
ULSHARED COUNTER BY
SETTING IT TO 9999





MAIN.C (DEMO) [1]



```
int main()
{
    UART_init();

    void priorityAndStackAttacker( void *pvParameters );
    void target( void *pvParameters );
    void globalVarAttacker( void *pvParameters );
    volatile uint32_t ulSharedCounter = 0;

    static TaskHandle_t globalVarAttackerHandle = NULL;

    xTaskCreate( priorityAndStackAttacker, "priorityAndStackAttacker",
                configMINIMAL_STACK_SIZE, NULL, 1, NULL );
    xTaskCreate( target, "target", configMINIMAL_STACK_SIZE, NULL, 1, NULL );
    xTaskCreate( globalVarAttacker, "globalVarAttacker",
                configMINIMAL_STACK_SIZE, NULL, 1, &globalVarAttackerHandle );

    vTaskStartScheduler();
    while( 1 );
}
```




MAIN.C (DEMO) [2]



```
void target( void *pvParameters )
{
    char pcTxBuffer[ 128 ];
    char pcSecret[ 65 ] = "ba7816bf8f01cfe (...);";
    char pcPassword[] = "SensitiveData123";
    char pcEncrypted[ 17 ]; /* same length as password (16 char + terminator) */

    while( 1 )
    {
        ulSharedCounter++; /* normal use */
        snprintf( pcTxBuffer, sizeof( pcTxBuffer ), "[target] Cycle %lu\n", ulSharedCounter );
        UART_printf( pcTxBuffer );

        for(uint16_t usIndex = 0; usIndex < sizeof( pcPassword ) - 1; usIndex++)
        { pcEncrypted[ usIndex ] = pcPassword[ usIndex ] ^ pcSecret[ usIndex % 64 ]; }
        pcEncrypted[ sizeof( pcPassword ) - 1 ] = '\0';
    }
    /* continues */
}
```



MAIN.C (DEMO) [3]



```
/* continues */
UART_printf( "[target] Encrypted: " ); /* Print the encrypted output in hexadecimal */
for( uint16_t usIndex = 0; usIndex < sizeof( pcPassword ) - 1; usIndex++ )
{
    snprintf( pcTxBuffer, sizeof( pcTxBuffer ), "%02x ", pcEncrypted[ usIndex ] );
    UART_printf( pcTxBuffer );
} UART_printf( "\n" );

if( strcmp( pcSecret, "ba7816bf8f01cfe (...)" ) == 0 ) /* Integrity check */
{ UART_printf( "[target] Key integrity OK\n" ); }
else
{ UART_printf( "[target] WARNING: Key corrupted! Respawning a clean target...\n" );
  ulSharedCounter = 0; xTaskCreate( target, (...) ); vTaskDelete( NULL );
}
vTaskDelay( pdMS_TO_TICKS( 500 ) );
}
```



MAIN.C (DEMO) [4]



```
void globalVarAttacker(void *pvParameters) {
char pcTxBuffer[ 128 ];

vTaskPrioritySet( NULL, configMAX_PRIORITIES - 3 ); /* higher than target, lower than
                                                    priorityAndStackAttacker */

while( 1 )
{
    ulTaskNotifyTake( pdTRUE, portMAX_DELAY ); /* Wait for notification from
                                                    priorityAndStackAttacker */

    ulSharedCounter = 9999; /* GLOBAL DATA ATTACK */
    snprintf( pcTxBuffer, sizeof( pcTxBuffer ), "\n[globalVarAttacker] GLOBAL DATA ATTACK:
                                                    ulSharedCounter forced to %lu\n\n\n", ulSharedCounter);

    UART_printf( pcTxBuffer );
}
}
```




MAIN.C (DEMO) [5]



```
void priorityAndStackAttacker( void *pvParameters )
{
    vTaskDelay( pdMS_TO_TICKS( 2000 ) ); /* wait for target to execute one correct cycle */
    vTaskPrioritySet( NULL, configMAX_PRIORITIES - 2 ); /* Increase priority */

    UBaseType_t uxNumberOfTasks; char pcTxBuffer[ 128 ];
    TaskStatus_t *pxTasksManaged = NULL; TaskStatus_t xCurrentTaskInfo;

    while( 1 )
    {
        uxNumberOfTasks = uxTaskGetNumberOfTasks();
        pxTasksManaged = pvPortMalloc( uxNumberOfTasks * sizeof( TaskStatus_t ) );

        if( pxTasksManaged != NULL ) {
            vTaskGetInfo( NULL, &xCurrentTaskInfo, pdFALSE, eInvalid );
            uxNumberOfTasks = uxTaskGetSystemState( pxTasksManaged, uxNumberOfTasks, NULL );
        }
        /* continues */
    }
}
```



MAIN.C (DEMO) [6]



```
/* continues */
for( UBaseType_t uxIndex = 0; uxIndex < uxNumberOfTasks; uxIndex++ )
{
    /* Inspect all external tasks */
    if( strcmp( pxTasksManaged[ uxIndex ].pcTaskName, xCurrentTaskInfo.pcTaskName ) != 0 )
    {
        /* extract and UART_printf base/top/end of each task */

        /* Pointers and counters for continuous tracking */
        uint8_t* pucCurrentPtr = pucEndOfStack - 1; /* Start from stack buffer's top byte */
        uint8_t* pucCurrentChainStart = NULL;
        uint16_t usCurrentChainLen = 0;

        char pcLongestString[ 256 ] = "";
        uint16_t usLongestLength = 0;
        uint8_t* pucLongestChainStartAddress = NULL;
    }
    /* continues */
}
```

IN TASK.H:

```
#if ( ( portSTACK_GROWTH > 0 ) || (
configRECORD_STACK_HIGH_ADDRESS == 1 ) )
    StackType_t * pxTopOfStack;
    StackType_t * pxEndOfStack;
#endif
```

SET TO 1 IN
FREERTOS
CONFIG.H



MAIN.C (DEMO) [7]



```
/* continues */
/* Scan byte by byte backwards down to the current stack pointer */
while(pucCurrentPtr >= pucTopOfStack) {
    uint8_t ucByteFromMem = *pucCurrentPtr;
    if( ucByteFromMem >= 32 && ucByteFromMem <= 122 ) {
        if( usCurrentChainLen == 0 )
            pucCurrentChainStart = pucCurrentPtr; /* First char found */
        usCurrentChainLen++;
    }
    else {
        /* Check if this printable chain is the longest found so far for this task */
        if( usCurrentChainLen > usLongestLength ) {
            /* save the current address, length and the actual longest string */
        }
        usCurrentChainLen = 0; /* Reset counter for the next potential string */
    } pucCurrentPtr--; /* Move down to the next byte */
} /* continues */
```

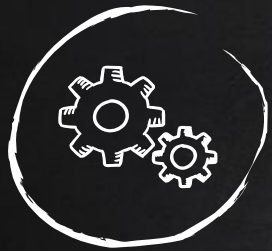


MAIN.C (DEMO) [8]



```
if( usLongestLength > 0 ) {
    /* print the longest string found */
    if( usLongestLength >= 32 && strcmp( pxTasksManaged[ uxIndex ].pcTaskName, "target" ) == 0 ) {
        /* Corrupt the string, inserting random values in its exact address */
    }
}
else {
    UART_printf( "[priorityAndStackAttacker] No printable strings found in this stack.\n" );
} } }
vPortFree( pxTasksManaged );
}

if ( globalVarAttackerHandle != NULL )
    xTaskNotifyGive( globalVarAttackerHandle ); /* Notify globalVarAttacker */
vTaskDelay( pdMS_TO_TICKS( 2000 ) ); /* Snapshot delay */
}
}
```



THE MPU PRINT ITERATION PROCESS

(TO AVOID THE USE OF SNPRINTF)

1.

SNPRINTF_MPU



A CUSTOM FUNCTION
WITH LOCAL VARIABLES
AND NO DYNAMIC
MEMORY ALLOCATIONS



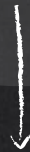
PERSONALIZED



COMPLEX, INEFFICIENT

2.

PRINT_HELPER



USES PRINT_INT,
PRINT_PTR AND PRINT_HEX
FUNCTIONS FOR PRINTS



CLEAN IMPLEMENTATION



REQUIRES THE USE OF
MULTIPLE FUNCTIONS

3.

NEWLIB
_SNPRINTF_R



REENTRANT VERSION OF
SNPRINTF WITH NO
GLOBAL OR DYNAMICALLY
ALLOCATED REFERENCES



CLEAN AND
STRAIGHTFORWARD



DEMO VS DEMO_MPU [1]: FREERTOSCONFIG.H AND MAKEFILE



```
#define configENABLE_MPU                1
#define configUSE_MPU_WRAPPERS_V1      ( 1 )
#define configENABLE_ACCESS_CONTROL_LIST ( 0 )
#define configPROTECTED_KERNEL_OBJECT_POOL_SIZE ( 150 )
#define configSYSTEM_CALL_STACK_SIZE   ( 128 )
#define configENFORCE_SYSTEM_CALLS_FROM_KERNEL_ONLY 1
#define configALLOW_UNPRIVILEGED_CRITICAL_SECTIONS 0



---


KERNEL_PORT_DIR      := $(KERNEL_DIR)/portable/GCC/ARM_CM3_MPU

# Additions in SOURCE_FILES:

SOURCE_FILES += $(KERNEL_DIR)/stream_buffer.c
SOURCE_FILES += $(KERNEL_DIR)/portable/GCC/ARM_CM3_MPU/port.c
SOURCE_FILES += $(KERNEL_DIR)/portable/Common/mpu_wrappers.c
```



DEMO VS DEMO_MPU [2]: MPS2_M3.LD (LINKER FILE)



```
/* Memory additions: */

__privileged_functions_region_size__ = 64K;
__privileged_data_region_size__      = 32K;
__unprivileged_data_region_size__    = 32K;

__privileged_data_start__ = __SRAM_segment_start__;
__privileged_data_end__   = __SRAM_segment_start__ +
                           __privileged_data_region_size__;

__privileged_functions_start__ = __FLASH_segment_start__;
__privileged_functions_end__   = __FLASH_segment_start__ +
                           __privileged_functions_region_size__;
```



DEMO VS DEMO_MPU [3]: MPS2_M3.LD (LINKER FILE #2)



```
.unprivileged_data :  
{  
    . = ALIGN(4);  
    __unprivileged_data_start__ = .;  
  
    *(.unprivileged_data)  
    *(.unprivileged_data*)  
  
    . = ALIGN(4);  
  
    FILL(0xDEAD);  
    . = __unprivileged_data_region_size__;  
  
    __unprivileged_data_end__ = .;  
} >RAM
```

```
.privileged_data :  
{  
    /* ... */  
} >RAM AT> FLASH  
  
.privileged_functions :  
{  
    /* ... */  
} >FLASH
```



DEMO VS DEMO_MPU [4]: STARTUP.C



```
void prvGetMPURegistersFromStack( uint32_t *pulFaultStackAddress );  
( uint32_t * ) &MemManage_Handler, // not Default_Handler anymore  
  
static void MemManage_Handler( void ) {  
    __asm volatile  
    (  
        ".align 8                                \n"  
        " tst lr, #4                             \n"  
        " ite eq                                 \n"  
        " mrseq r0, msp                          \n"  
        " mrsne r0, psp                          \n"  
        " ldr r2, =prvGetMPURegistersFromStack   \n"  
        " bx r2                                  \n"  
        " .ltorg                                 \n"  
    ); }
```



DEMO VS DEMO_MPU [5]: STARTUP.C (#2)



```
__attribute__( ( used ) ) void prvGetMPURegistersFromStack( uint32_t *pulFaultStackAddress )
{
    #define portNVIC_MMFSR_REG    ( * ( ( volatile uint8_t * ) 0xE000ED28 ) )
    #define portNVIC_MMFAR_REG    ( * ( ( volatile uint32_t * ) 0xE000ED34 ) )
    uint8_t ucMMFSR = portNVIC_MMFSR_REG; /*Error Type*/
    uint32_t ulMMFAR = portNVIC_MMFAR_REG; /*Violated Address*/
    char logBuffer[128];

    UART_printf("!!! MPU VIOLATION DETECTED: MemManage Fault !!!\r\n");
    snprintf(logBuffer, sizeof(logBuffer), "Instruction pointer (PC): 0x%08X\r\n", (unsigned int)pc);
    UART_printf(logBuffer);

    /* ... Continues ... */
}
```




DEMO VS DEMO_MPU [6]: STARTUP.C (#3)



```
/* ... Continues ... */

if( ( ucMMFSR & 0x02 ) != 0 ) {
    snprintf(logBuffer, sizeof(logBuffer), "Violated Memory Address: 0x%08X\r\n", (unsigned nt)uIMMFAR);
    UART_printf(logBuffer);
} else {
    UART_printf("Violated Memory Address: Non-aliased or hidden access.\r\n");
}

/* Cause's detailed analysis */
if( ( ucMMFSR & 0x01 ) != 0 ) UART_printf("Violation Type: IACCVIOL (Instruction Fetch Violation)\r\n");
if( ( ucMMFSR & 0x02 ) != 0 ) UART_printf("Violation Type: DACCVIOL (Data Read/Write Violation)\r\n");

for( ;; );
}
```



MAIN.C (DEMO_MPU) [1]



```
#define STACK_SIZE_WORDS ( configMINIMAL_STACK_SIZE )
#define STACK_SIZE_BYTES ( STACK_SIZE_WORDS * sizeof( StackType_t ) )

static StackType_t priorityAndStackAttackerStack[ STACK_SIZE_WORDS ]
    __attribute__( ( section( ".unprivileged_data" ), aligned( STACK_SIZE_BYTES ) ) );
    /* defining the memory for the priorityAndStackAttacker stack */

static StackType_t targetStack[ STACK_SIZE_WORDS ]
    __attribute__( ( section( ".unprivileged_data" ), aligned( STACK_SIZE_BYTES ) ) );
    /* defining the memory for the target stack */

static StackType_t globalVarAttackerStack[ STACK_SIZE_WORDS ]
    __attribute__( ( section( ".unprivileged_data" ), aligned( STACK_SIZE_BYTES ) ) );
    /* defining the memory for the globalVarAttacker stack */

volatile uint32_t ulSharedCounter __attribute__( ( aligned( 32 ) ) ) = 0;
```



MAIN.C (DEMO_MPU) [2]



```
int main()
{
    /* ... */
    TaskParameters_t targetParams = {
        .pvTaskCode = target, .pcName = "target", .usStackDepth = STACK_SIZE_WORDS,
        .pvParameters = NULL, .uxPriority = 1, .puxStackBuffer = targetStack,
        .xRegions =
        {
            { ( void* )&ulSharedCounter, 32, portMPU_REGION_READ_WRITE },
                /* allows target to have access to the global variable */
            { 0, 0, 0 },
            { 0, 0, 0 } /* unused region */
        }
    };

    /* continues with globalVarAttackerParams and priorityAndStackAttackerParams */
}
```



MAIN.C (DEMO_MPU) [3]



```
/* Use of _sn_printf_r inside of void target( void *pvParameters ): */

struct _reent xLocalReent;
    _REENT_INIT_PTR(&xLocalReent);    /* Static Inizialization for _snprintf_r */

_snprintf_r( &xLocalReent, pcTxBuffer, sizeof( pcTxBuffer ), "[target] Cycle %lu\n",
                                                    (unsigned long)ulSharedCounter );
UART_printf( pcTxBuffer );

for( uint16_t usIndex = 0; usIndex < sizeof( password ) - 1; usIndex++ )
{
    _snprintf_r( &xLocalReent, pcTxBuffer, sizeof( pcTxBuffer ), "%02x ",
                                                    encrypted[ usIndex ] );
    UART_printf( pcTxBuffer );
}
UART_printf( "\n" );
```



MAIN.C (DEMO_MPU) [4]



```
void globalVarAttacker( void *pvParameters )
{
    ( void ) pvParameters;
    vTaskDelay( pdMS_TO_TICKS( 3000 ) ); /* To allow target to set ulSharedCounter */

    vTaskPrioritySet( NULL, configMAX_PRIORITIES - 2 );
    UART_printf(" \n\n[globalVarAttacker] globalVarAttacker priority increased!\n" );

    ulSharedCounter = 9999; /* MemManage Fault (globalVarAttacker is unprivileged */
    _snprintf_r( &xLocalReent, pcTxBuffer, sizeof( pcTxBuffer ), "[globalVarAttacker]
        Attack successful! ulSharedCounter forced to %lu\n", ulSharedCounter );
    UART_printf( pcTxBuffer );

    while( 1 )
    { vTaskDelay(pdMS_TO_TICKS(1000)); }
}
```




MAIN.C (DEMO_MPU) [5]



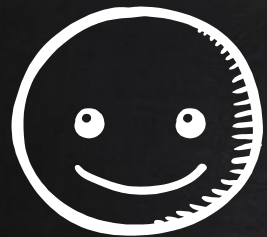
```
/* Differences compared to priorityAndStackAttacker without MPU: */

void priorityAndStackAttacker( void *pvParameters )
{
    TaskStatus_t pxTasksManaged[ 10 ]; /* Now statically allocated */

    struct _reent xLocalReent;
    _REENT_INIT_PTR(&xLocalReent); /* Static Inizialization for _snprintf_r */

    _snprintf_r( &xLocalReent, pcTxBuffer, sizeof( pcTxBuffer ), "\n+++ Analyzing
                    Stack for Task: %s +++\n", pxTasksManaged[ uxIndex ].pcTaskName );
    UART_printf( pcTxBuffer );

    /* No notification given to globalVarAttacker anymore */
}
```



THANK YOU FOR
YOUR ATTENTION



NOÈ GABRIELE - s364766

PITTALIS LUCA - s358755

PRINCIVALLE SERGIO - s364116

RUSSO LORENZO - s358556

